

Proceso de Creación del Bot de Telegram IA

Autor: Izan Urios

Fecha: Mayo 2026

Descripción: Documento que detalla todo el proceso de creación del bot, desde la preparación del sistema hasta su implementación final con OpenCode.

1. Formateo del Ordenador e Instalación de Linux Mint

1.- Prompt:

"Oye, quiero formatear mi ordenador e instalarle Linux Mint desde cero. Tengo un USB, ¿cómo puedo hacerlo paso a paso?"

2.- Resultado:

Se creó un USB bootable con Linux Mint usando herramientas como Rufus o Etcher. Se procedió al formateo completo del disco duro e instalación limpia de Linux Mint en el sistema.

3.- Problemas que ocasionó:

- Pérdida de datos anteriores en el disco duro (solucionado haciendo backup previo). - Configuración inicial de drivers y periféricos tras la instalación.

4.- Prompt que lo solucionó:

"Acabo de instalar Linux Mint y hay cosas que no van bien. ¿Me ayudas a configurar los drivers y dejar el sistema a punto para programar?"

2. Instalación de OpenCode

1.- Prompt:

"Quiero instalar OpenCode en mi Linux Mint. ¿Me dices los comandos y cómo comprobar que funciona bien?"

2.- Resultado:

Se instaló OpenCode mediante los comandos apropiados para Linux Mint (basado en Ubuntu). Se verificó la instalación ejecutando `opencode --version` y configurando el entorno inicial.

3.- Problemas que ocasionó:

- Faltaban algunas dependencias en el sistema base de Linux Mint.

4.- Prompt que lo solucionó:

"OpenCode parece que necesita más cosas en mi sistema. ¿Me instalas lo que falte para que funcione bien?"

3. Creación del Bot de Telegram con OpenRouter y OpenClaw

3.1. Prompt Inicial - Creación del Bot Básico

1.- Prompt:

"Escucha, quiero hacer un bot de Telegram que sea inteligente. Necesito que me hagas uno que:

- *Se conecte a OpenRouter para usar modelos de IA (como GPT-4o-mini)*
- *Que pueda hablar conmigo de forma natural*
- *Que si le pregunto algo que requiera búsqueda (noticias, precios, clima), busque en internet automáticamente*
- *Que use DuckDuckGo para buscar (que no necesite API key)*
- *Que me pueda decir el clima de cualquier ciudad usando wtr.in*
- *Que me diga la hora y fecha actual de Madrid*
- *Que tenga comandos básicos: /start, /help, /hora, /clima, /status*
- *Que las claves (Telegram y OpenRouter) las ponga en variables de entorno, no directamente en el código*
- *Que el código sea limpio y fácil de entender*
- *Que me responda con emojis y formato bonito (Markdown)*

¿Puedes hacerme el código completo en un archivo `telegram_bot.py` para poder ejecutarlo ya?"

2.- Resultado:

Se generó el archivo `telegram_bot.py` con una implementación completa que incluye: - Integración con Telegram Bot API usando python-telegram-bot - Conexión a OpenRouter para modelos LLM (GPT-4o-mini) - Búsqueda web automática con DuckDuckGo cuando se detecta una pregunta - Consulta de clima con wtr.in (sin necesidad de API key) - Comandos: /start, /help, /status, /hora, /clima - Detección automática de preguntas sobre fecha/hora - Manejo de errores y logging - Variables de entorno para las claves (usando os.getenv)

3.- Problemas que ocasionó:

- Al principio las API keys se pusieron directamente en el código (problema de seguridad).

4.- Prompt que lo solucionó:

"Oye, acabo de ver que las claves del bot y de OpenRouter están puestas directamente en el código en lugar de usar variables de entorno. ¿Me lo arreglas para que las lea con `os.getenv`? Ah, y hazme un archivo `.env.example` para saber qué tengo que poner. Por cierto, asegúrate de que el `.gitignore` tenga el `.env` para no subir las claves a GitHub."

3.2. Prompt - Mejoras en la Detección de Intenciones

1.- Prompt:

"El bot funciona bien, pero quiero que sea más inteligente detectando cuándo tiene que buscar en internet. ¿Podrías añadirle una lista más completa de palabras clave para que sepa cuándo buscar? Cosas como 'noticias', 'precio', 'actual', 'reciente', 'cuánto', 'qué pasó', etc.

También que detecte cuando pregunto por la hora o la fecha, para responderme directamente sin pasar por la IA.

Ah, y que si le escribo 'qué hora es' o 'qué día es hoy' me lo diga sin buscar en internet."

2.- Resultado:

Se mejoró el bot con: - Lista ampliada de `SEARCH_KEYWORDS` para detectar cuándo buscar en internet - Lista de `TIME_DATE_KEYWORDS` para detectar preguntas sobre hora/fecha - Detección automática que responde directamente a preguntas de tiempo sin usar IA ni búsqueda

3.3. Prompt - Mejorar las Respuestas de la IA

1.- Prompt:

"Quiero que el bot sea más útil cuando responde con IA. ¿Podrías hacer que:

- Le pase la fecha y hora actual a la IA para que sepa qué día es
- Le diga a la IA que SI le pasamos resultados de búsqueda web, ESA es su única fuente de información (que no diga que no tiene información reciente)
- Que las respuestas sean concisas y usen emojis
- Que si la respuesta no tiene un emoji al principio, le ponga uno (🤖)"

2.- Resultado:

Se mejoraron las respuestas de la IA con: - Fecha y hora actual pasada en el system prompt - Instrucción obligatoria para la IA sobre el uso de resultados de búsqueda - Formato consistente con emojis en todas las respuestas

3.4. Prompt - Scripts de Utilidad

1.- Prompt:

"Para terminar, hazme unos scripts que me ayuden a gestionar el bot:

- 1. Un `run_bot.sh` que asegure que solo hay una instancia del bot corriendo (usando lock files para que no se ejecuten dos a la vez)*
- 2. Un `remote_control.sh` para gestionar el bot remotamente*
- 3. Un `requirements.txt` con todas las dependencias necesarias*
- 4. Un `README.md` con badges, tablas y ejemplos de uso*

Que todo esté en español y con buena pinta."

2.- Resultado:

Se generaron todos los scripts y documentación solicitada con un nivel de calidad profesional.

Resumen del Proceso

Todo el desarrollo del bot se realizó mediante prompts a OpenCode, sin programar nada manualmente. El proceso siguió estos pasos:

- 1. Preparación del entorno:** Formateo e instalación de Linux Mint
- 2. Herramientas:** Instalación de OpenCode
- 3. Desarrollo del bot:** Creación del bot con funcionalidades de IA, búsqueda web y clima
- 4. Mejoras iterativas:** Ajustes en la detección de intenciones y respuestas
- 5. Documentación y scripts:** Generación de README, requirements.txt y scripts de utilidad

Nota importante: Ningún código fue escrito manualmente. Todo fue generado por OpenCode siguiendo las indicaciones dadas en los prompts.

Autor: Izan Urios

Fecha de documentación: Mayo 2026

Herramienta utilizada: OpenCode AI (opencode.ai)